Lecture 12: RNLA Case Study

Jamie Haddock

Table of contents

0.1 Case study: randomness as a tool for linear algebra 1

1 Part A: Randomized Kaczmarz **1**
1.1 Collision probability in randomized Kaczmarz 2
1.2 LOTUS: why the row-norm PMF? 4
1.3 Linearity + LOTUS: average one-step progress of RK 4
1.4 Watching the bound play out over many steps 5
1.5 Recursing the randomized Kaczmarz bound 6
1.6 Uniform vs. row-norm sampling: the distribution controls the rate 6

2 Part B: Randomized trace estimation **8**
2.1 Example: unbiasedness of randomized trace estimation 8
2.2 Example: continuous RV in randomized trace estimation 9
2.3 How much variance? The Rademacher estimator 10
2.4 Comparing the two estimators 10

3 Part C: Randomized matrix multiplication **11**
3.1 Unbiasedness of randomized matrix multiplication 11
3.2 The best sampling distribution 12

0.1 Case study: randomness as a tool for linear algebra

Three randomized algorithms from **randomized numerical linear algebra (RNLA)**:

1. **Randomized Kaczmarz** – solve $Ax = b$ by projecting onto one randomly chosen row at a time.
2. **Randomized trace estimation** – estimate $\mathrm{tr}(A)$ using only products $z^\top Az$ with a random vector z.
3. **Randomized matrix multiplication** – approximate AB using a few randomly chosen column/row outer products.

> The common story
>
> Each method returns a *random* answer whose **expected value is either the exact, or an improved answer**.

1 Part A: Randomized Kaczmarz

Randomized Kaczmarz (RK) solves a consistent linear system $Ax = b$, where $A \in \mathbb{R}^{m\times n}$ has rows $a_1^\top, \dots, a_m^\top$ and $x^\star$ denotes a solution. Starting from an initial guess x_0 (for instance $x_0 = 0$), at each iteration k the algorithm:

1. **Samples a row.** Draw a row index $I_k \in \{1, \dots, m\}$ at random from a fixed PMF $p = (p_1, \dots, p_m)$, independently of all previous draws (sampling **with replacement**).

2. **Projects onto that equation.** Move the current iterate to the closest point on the hyperplane $\{x : a_{I_k}^\top x = b_{I_k}\}$:

$$x_{k+1} = x_k + \frac{b_{I_k} - a_{I_k}^\top x_k}{\|a_{I_k}\|^2} a_{I_k}.$$

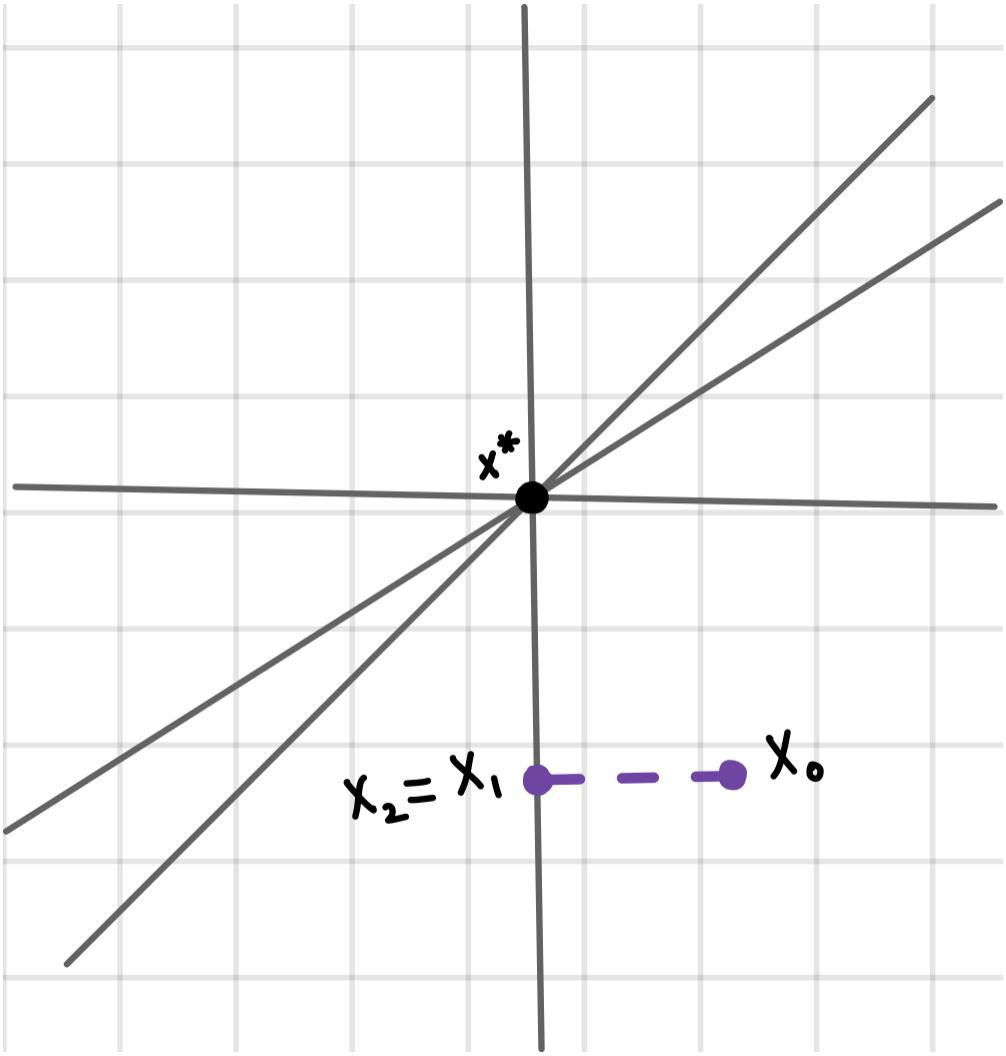
After the kth step the iterate satisfies equation I_k exactly, so sampling the same row twice in a row (a **collision**) makes no progress.

Two standard choices of PMF are the **row-norm** PMF $p_i = \|a_i\|^2/\|A\|_F^2$ and the **uniform** PMF $p_i = 1/m$.

> Notation: the Frobenius norm
>
> For a matrix $A \in \mathbb{R}^{m\times n}$ with entries A_{ij} and rows $a_1^\top, \dots, a_m^\top$, the **Frobenius norm** is
>
> $$\|A\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} A_{ij}^2} \quad \Longrightarrow \quad \|A\|_F^2 = \sum_{i=1}^{m} \|a_i\|^2.$$

1.1 Collision probability in randomized Kaczmarz

Suppose two consecutive iterations sample rows I_1, I_2 **independently**, each $\sim p$ (with replacement). What's the probability the algorithm samples the **same row twice in a row** – a *collision*?



Theorem: Collision probability

$$P(I_1 = I_2) = \sum_{i=1}^{m} p_i^2.$$

Proof:

Condition on I_1, applying the law of total probability with partitioning on the value of I_1:

$$P(I_1 = I_2) = \sum_{i=1}^{m} P(I_2 = i \mid I_1 = i)\, P(I_1 = i) = \sum_{i=1}^{m} P(I_2 = i)\, p_i = \sum_{i=1}^{m} p_i^2,$$

using independence of I_1, I_2 for $P(I_2 = i \mid I_1 = i) = P(I_2 = i) = p_i$. ■

Why we care: if row i is sampled twice in a row, the second projection doesn't update the iterate – the iterate already satisfies $a_i^\top x = b_i$ exactly after the first projection, so the second step makes *zero* progress toward $x^\star$. A high collision probability means wasted iterations.

Fact

The collision probability $\sum_i p_i^2$ is **minimized** by the *uniform* distribution $p_i = 1/m$ (giving $\sum_i p_i^2 = 1/m$, the smallest possible value: $1 = (\sum_i p_i)^2 \le m \sum_i p_i^2$ by Cauchy-Schwarz), and **maximized** at 1 when all the probability mass sits on a single row. So concentrating the sampling distribution on a subset of rows comes at the cost of more wasted iterations.

Exercise 1: Bayes' rule on collisions

Sample I_1, I_2 independently from a PMF p on $\{1, \dots, m\}$, and let $C = \{I_1 = I_2\}$ be the collision event.

(a) Use Bayes' rule and the collision theorem to show that

$$P(I_1 = i \mid C) = \frac{p_i^2}{\sum_{j=1}^m p_j^2}.$$

(b) Now take $m = 3$ and $p = (\frac{1}{2}, \frac{1}{3}, \frac{1}{6})$. Compute $P(C)$, and compute $P(I_1 = i \mid C)$ for $i = 1, 2, 3$. Compare each with the unconditional probability $P(I_1 = i) = p_i$. For which rows is the conditional probability larger, and for which is it smaller? Explain in words why a collision makes the heavily weighted row even more likely.

1.2 LOTUS: why the row-norm PMF?

A popular distribution over the rows is the **row-norm PMF** $p_I(i) = \|a_i\|^2 / \|A\|_F^2$. We'll see why now!

Fix the current iterate x_k and let $v = x_k - x^\star$. LOTUS lets us compute the expected value of *any* function of I straight from p_I – for instance

$$E\left[\frac{(a_I^\top v)^2}{\|a_I\|^2}\right] = \sum_{i=1}^m \frac{(a_i^\top v)^2}{\|a_i\|^2} p_I(i).$$

Plugging in $p_I(i) = \|a_i\|^2 / \|A\|_F^2$, the $\|a_i\|^2$ cancels:

$$\sum_{i=1}^m \frac{(a_i^\top v)^2}{\|a_i\|^2} \cdot \frac{\|a_i\|^2}{\|A\|_F^2} = \frac{1}{\|A\|_F^2} \sum_{i=1}^m (a_i^\top v)^2 = \frac{\|Av\|^2}{\|A\|_F^2}.$$

If I were sampled **uniformly** instead, the $\|a_i\|^2$ would *not* cancel, and the final form wouldn't be so nice and simple!

1.3 Linearity + LOTUS: average one-step progress of RK

Recall randomized Kaczmarz's update, and note that the Pythagorean theorem (the update is an orthogonal projection) provides,

$$\|x_{k+1} - x^\star\|^2 = \|v\|^2 - \frac{(a_I^\top v)^2}{\|a_I\|^2}.$$

where $v = x_k - x^\star$.

$\|v\|^2$ is a **constant** given x_k – only I is random. By linearity,

$$E[\|x_{k+1} - x^\star\|^2] = \|v\|^2 - E\left[\frac{(a_I^\top v)^2}{\|a_I\|^2}\right] = \|v\|^2 - \frac{\|Av\|^2}{\|A\|_F^2},$$

reusing the LOTUS computation from the previous slide!

Since $\|Av\| \geq \sigma_{\min}(A)\|v\|$,

$$E[\|x_{k+1} - x^\star\|^2] \;\leq\; \left(1 - \frac{\sigma_{\min}(A)^2}{\|A\|_F^2}\right) \|x_k - x^\star\|^2.$$

Every step shrinks the *expected* squared error by a fixed factor – this a very nice (and very well-cited) result from Thomas Strohmer and Roman Vershynin in 2009!

1.4 Watching the bound play out over many steps

The one-step bound says the expected squared error shrinks by a fixed factor *every* step. Let's actually run RK and watch $\|x_k - x^\star\|^2$ decrease and compare it to the bound.

```
import numpy as np
import matplotlib.pyplot as plt

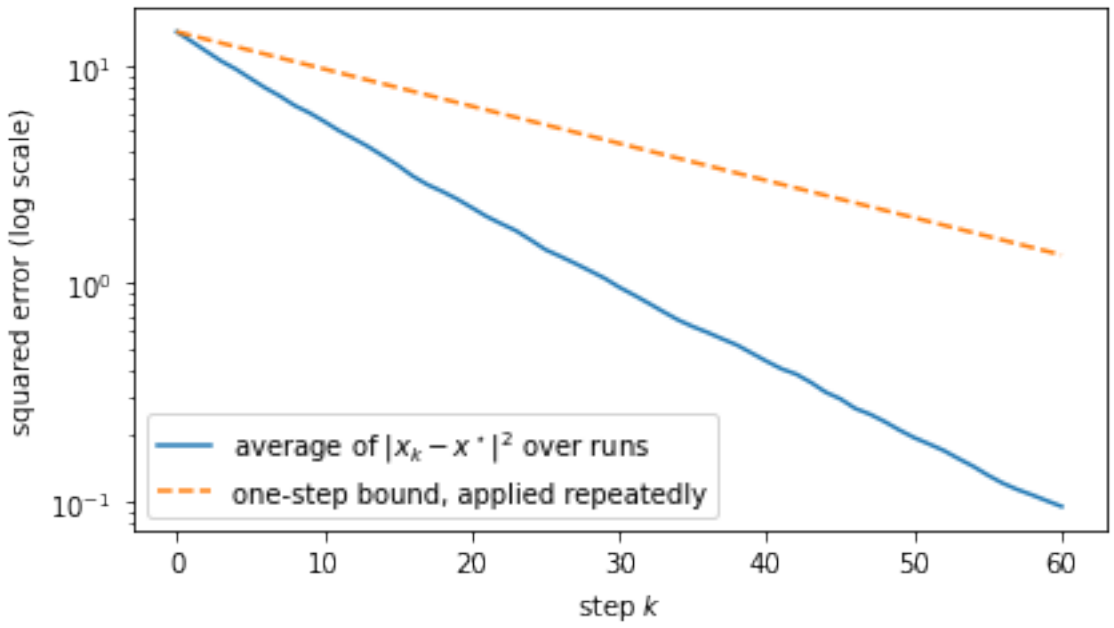
rng = np.random.default_rng(151)
m, n = 20, 8
A = rng.standard_normal((m, n))
x_star = rng.standard_normal(n)
b = A @ x_star
row_norm_sq = (A ** 2).sum(axis=1)
p_row_norm = row_norm_sq / row_norm_sq.sum()
sigma_min = np.linalg.svd(A, compute_uv=False)[-1]
frob_sq = row_norm_sq.sum()

num_steps = 60
num_runs = 300
errors_sq = np.zeros((num_runs, num_steps + 1))

for r in range(num_runs):
    x = np.zeros(n)
    errors_sq[r, 0] = np.linalg.norm(x - x_star) ** 2
    idx_sequence = rng.choice(m, size=num_steps, p=p_row_norm)
    for t, i in enumerate(idx_sequence, start=1):
        a_i, b_i = A[i], b[i]
        x = x + (b_i - a_i @ x) / row_norm_sq[i] * a_i
        errors_sq[r, t] = np.linalg.norm(x - x_star) ** 2

mean_error_sq = errors_sq.mean(axis=0)
bound_curve = errors_sq[:, 0].mean() * (1 - sigma_min**2 / frob_sq) ** np.arange(num_steps + 1)

fig, ax = plt.subplots(figsize=(6, 3.5))
ax.semilogy(mean_error_sq, label="average of $\\|x_k-x^\\star\\|^2$ over runs")
ax.semilogy(bound_curve, "--", label="one-step bound, applied repeatedly")
ax.set_xlabel("step $k$")
ax.set_ylabel("squared error (log scale)")
ax.legend()
plt.tight_layout()
plt.show()
```



1.5 Recursing the randomized Kaczmarz bound

Recall our previous one-step bound, valid conditional on the *current* iterate x_k:

$$E[\|x_{k+1} - x^\star\|^2 \mid x_k] \le (1-c)\,\|x_k - x^\star\|^2, \qquad c = \frac{\sigma_{\min}(A)^2}{\|A\|_F^2}.$$

That's only a statement about *one* step, conditional on where we happen to be. To turn it into an unconditional guarantee after k steps, apply the law of iterated expectation repeatedly:

$$\begin{aligned} E[\|x_2 - x^\star\|^2] &= E\Big[E[\|x_2 - x^\star\|^2 \mid x_1]\Big] \le E[(1-c)\|x_1 - x^\star\|^2] \\ &= (1-c)\,E[\|x_1 - x^\star\|^2] \le (1-c)^2\|x_0 - x^\star\|^2. \end{aligned}$$

Repeating this argument (this is called the *tower of iterated expectation*) gives, for every k,

$$E[\|x_k - x^\star\|^2] \le (1-c)^k\|x_0 - x^\star\|^2,$$

– exactly the compounded bound we *plotted* before. Now we know why "applying the one-step bound repeatedly" is legitimate: it's the law of total expectation, used k times.

1.6 Uniform vs. row-norm sampling: the distribution controls the rate

One can prove a similar (but worse) bound for the error for uniform sampling. Let's run both samplers on a matrix whose row norms vary by orders of magnitude, and compare each empirical curve with its bound.

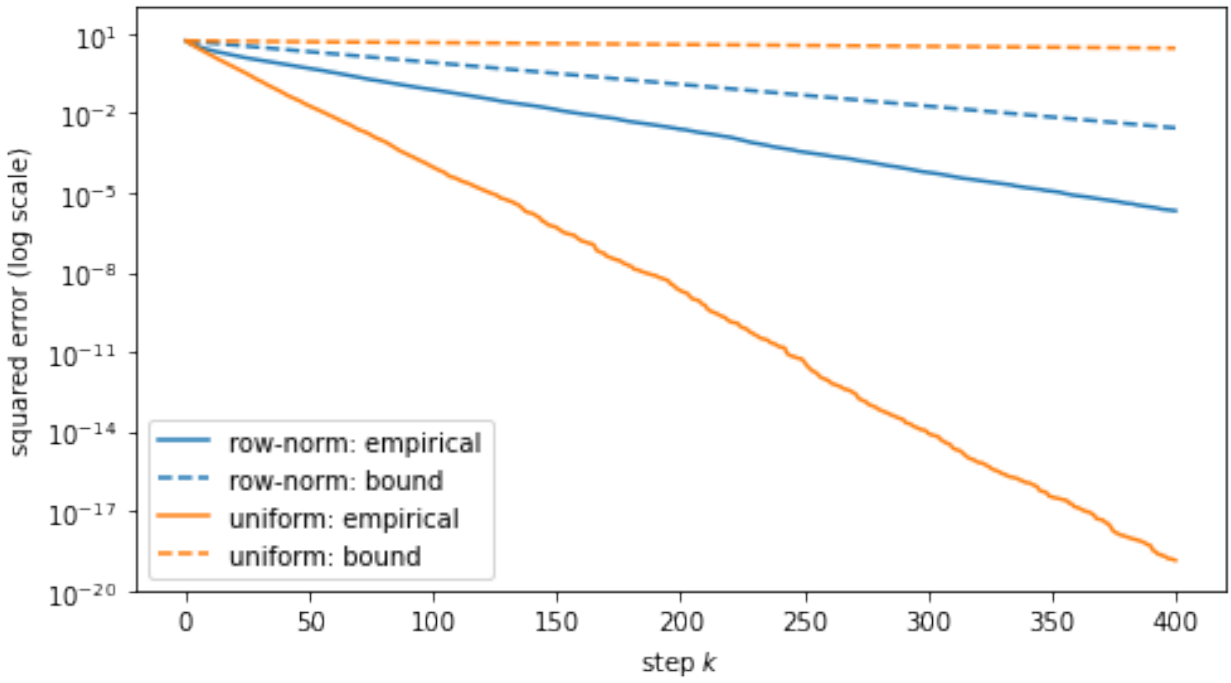
```
rng = np.random.default_rng(151)
m, n = 40, 8
A = rng.standard_normal((m, n)) * (10 ** rng.uniform(-1, 1, size=(m, 1)))  # very unequal row norms
x_star = rng.standard_normal(n)
b = A @ x_star
row_norm_sq = (A ** 2).sum(axis=1)
frob_sq = row_norm_sq.sum()
```

```
sigma_min = np.linalg.svd(A, compute_uv=False)[-1]

samplers = {
    "row-norm": row_norm_sq / frob_sq,
    "uniform": np.ones(m) / m,
}
rates = {
    "row-norm": sigma_min**2 / frob_sq,
    "uniform": sigma_min**2 / (m * row_norm_sq.max()),
}

num_steps, num_runs = 400, 300
mean_err = {}
for name, p in samplers.items():
    errs = np.zeros((num_runs, num_steps + 1))
    for r in range(num_runs):
        x = np.zeros(n)
        errs[r, 0] = np.linalg.norm(x - x_star) ** 2
        for t, i in enumerate(rng.choice(m, size=num_steps, p=p), start=1):
            x = x + (b[i] - A[i] @ x) / row_norm_sq[i] * A[i]
            errs[r, t] = np.linalg.norm(x - x_star) ** 2
    mean_err[name] = errs.mean(axis=0)

e0 = np.linalg.norm(x_star) ** 2
ks = np.arange(num_steps + 1)
fig, ax = plt.subplots(figsize=(7, 4))
for name, color in [("row-norm", "C0"), ("uniform", "C1")]:
    ax.semilogy(mean_err[name], color=color, label=f"{name}: empirical")
    ax.semilogy(e0 * (1 - rates[name]) ** ks, "--", color=color, label=f"{name}: bound")
ax.set_ylim(1e-20, 1e2)
ax.set_xlabel("step $k$")
ax.set_ylabel("squared error (log scale)")
ax.legend()
plt.tight_layout()
plt.show()
```



Takeaways

- The sampling distribution controls the **guarantee**: row-norm sampling gives a much better bound here.
- The bounds are only *upper* bounds. On this matrix uniform sampling actually converges *faster* in practice – row-norm sampling spends most of its draws on a few huge rows, while uniform sampling touches every row equally often. A bound being better does not mean the algorithm is better on a given matrix.

Exercise 2: distinct rows

Run RK with replacement for k steps, and let D_k be the number of **distinct** rows sampled.

(a) Write $D_k = \sum_{i=1}^{m} Y_i$ for indicator RVs Y_i, and say what event Y_i indicates.

(b) Use independence of the draws to compute $P(Y_i = 1)$, then use linearity to show

$$E[D_k] = \sum_{i=1}^{m} (1 - (1 - p_i)^k).$$

Check the case $k = 1$.

2 Part B: Randomized trace estimation

Recall the randomized trace estimator: for $A \in \mathbb{R}^{n \times n}$, draw $z \in \{-1, +1\}^n$ with i.i.d. Rademacher entries and set $\tau = z^\top A z = \sum_{i,j} z_i z_j A_{ij}$.

2.1 Example: unbiasedness of randomized trace estimation

We'll now prove $E[\tau] = \mathrm{tr}(A)$ – recall this is called **unbiasedness**.

By linearity, $E[\tau] = \sum_{i,j} A_{ij}\, E[z_i z_j]$. Split into diagonal and off-diagonal terms:

- **Diagonal ($i = j$):** $z_i^2 = 1$ always, so $E[z_i^2] = 1$.
- **Off-diagonal ($i \neq j$):** z_i, z_j independent, so by the fact above, $E[z_i z_j] = E[z_i]E[z_j] = 0 \cdot 0 = 0$ (Rademacher has mean 0).

$$E[\tau] = \sum_i A_{ii} \cdot 1 + \sum_{i \neq j} A_{ij} \cdot 0 = \sum_i A_{ii} = \mathrm{tr}(A).$$

Exercise 3: breaking the proof

Suppose that instead of independent entries, we use a single random sign $s \in \{-1, +1\}$ (each with probability 1/2) and set $z = s\mathbf{1} = (s, s, \ldots, s)$.

(a) Check that each z_i still has mean 0 and variance 1.
(b) Compute $E[z^\top A z]$. What do you get instead of $\mathrm{tr}(A)$?
(c) Which step of the unbiasedness proof fails, and why?

2.2 Example: continuous RV in randomized trace estimation

The unbiasedness proof only ever used

$$E[z_i^2] = 1 \qquad \text{and} \qquad E[z_i z_j] = 0 \ (i \neq j),$$

which followed from mean 0, variance 1, and independence – **nothing about z_i being discrete**. So a continuous z_i works exactly as well.

A continuous alternative

Instead of Rademacher entries $z_i = \pm 1$, draw z_i i.i.d. from a continuous distribution with mean 0 and variance 1 – the standard normal $N(0, 1)$ (the Gaussian). The same proof works exactly, just a different distribution!

```
rng = np.random.default_rng(151)
d = 50
A = rng.standard_normal((d, d)) + 5 * np.eye(d)
true_trace = np.trace(A)

num_samples = 20_000
Z_rad = rng.choice([-1.0, 1.0], size=(num_samples, d))  # discrete: Rademacher
Z_gauss = rng.standard_normal((num_samples, d))          # continuous: standard Gaussian

tau_rad = np.einsum('si,ij,sj->s', Z_rad, A, Z_rad)
tau_gauss = np.einsum('si,ij,sj->s', Z_gauss, A, Z_gauss)

print(f"true trace(A)                          = {true_trace:.4f}")
print(f"mean of tau, Rademacher z (discrete)  = {tau_rad.mean():.4f}")
print(f"mean of tau, Gaussian z (continuous)  = {tau_gauss.mean():.4f}")
```

```
true trace(A)                          = 250.0783
mean of tau, Rademacher z (discrete)  = 249.9342
mean of tau, Gaussian z (continuous)  = 249.5031
```

2.3 How much variance? The Rademacher estimator

Unbiased is not enough – a single sample of τ could be far from $\mathrm{tr}(A)$. Let A be **symmetric**, so $A_{ij} = A_{ji}$ and

$$\tau = \sum_i A_{ii} z_i^2 + 2 \sum_{i<j} A_{ij} z_i z_j = \mathrm{tr}(A) + 2 \sum_{i<j} A_{ij} z_i z_j$$

for Rademacher z (since $z_i^2 = 1$). The first term is a constant, so only the cross terms contribute to the variance.

Variance of the Rademacher estimator. For symmetric A, we have

$$\mathrm{Var}(\tau) = 4 \,\mathrm{Var}\Big(\sum_{i<j} A_{ij}\, z_i z_j \Big).$$

Let $W_{ij} = z_i z_j$ for $i < j$. We check that these are **uncorrelated with variance 1**:

- **Mean zero.** $E[W_{ij}] = E[z_i]E[z_j] = 0$ by independence.
- **Variance one.** $\mathrm{Var}(W_{ij}) = E[z_i^2 z_j^2] - 0 = 1$, since $z_i^2 = z_j^2 = 1$.
- **Pairwise uncorrelated.** For distinct pairs $\{i,j\} \neq \{k,l\}$, the product $W_{ij}W_{kl}$ has at least one index appearing exactly once. On the other hand, if $j = k$ then $W_{ij}W_{jl} = z_i z_j^2 z_l = z_i z_l$ with $i \neq l$. So $E[W_{ij}W_{kl}]$ is a product of terms including $E[z_m] = 0$, and $\mathrm{Cov}(W_{ij}, W_{kl}) = 0$.

(They are *not* mutually independent: $W_{12}W_{13}W_{23} = 1$ always. We only need uncorrelated, since variances add for uncorrelated summands.)

So variances of the sum add:

$$\mathrm{Var}(\tau) = 4 \sum_{i<j} A_{ij}^2 \,\mathrm{Var}(W_{ij}) = 4 \sum_{i<j} A_{ij}^2 = 2 \sum_{i \neq j} A_{ij}^2,$$

where the last step uses $\sum_{i \neq j} A_{ij}^2 = 2 \sum_{i<j} A_{ij}^2$ by symmetry.

> **Diagonal matrices**
>
> If A is diagonal, $\mathrm{Var}(\tau) = 0$: **every** draw of z gives exactly $\mathrm{tr}(A)$. The variance only measures the off-diagonal mass.

> **Exercise 4: variance of the Gaussian estimator**
>
> Let A be symmetric and z have i.i.d. $N(0,1)$ entries. You may use $E[z_i^4] = 3$.
> (a) Show $\mathrm{Var}(z_i^2) = 2$.
> (b) Write $\tau = \sum_i A_{ii} z_i^2 + 2 \sum_{i<j} A_{ij} z_i z_j$. Argue that all terms are pairwise uncorrelated, and conclude that
>
> $$\mathrm{Var}(\tau_{\mathrm{Gauss}}) = 2 \|A\|_F^2.$$

2.4 Comparing the two estimators

Since $\|A\|_F^2 = \sum_i A_{ii}^2 + \sum_{i \neq j} A_{ij}^2$, the two variances differ by

$$\mathrm{Var}(\tau_{\mathrm{Gauss}}) - \mathrm{Var}(\tau_{\mathrm{Rad}}) = 2 \sum_i A_{ii}^2 \;\geq 0.$$

Rademacher always has smaller variance, and it wins by the most when A has a heavy diagonal. If A has zero diagonal, the two are identical. Let's check on a diagonal-dominant matrix and on one with zero diagonal.

```python
rng = np.random.default_rng(151)
d = 30
G = rng.standard_normal((d, d))
S = (G + G.T) / 2                                  # symmetric, generic diagonal
A_diag_heavy = S + 5 * np.eye(d)                   # diagonal-dominant
A_zero_diag = S - np.diag(np.diag(S))              # zero diagonal

num_samples = 200_000
for name, M in [("diagonal-dominant", A_diag_heavy), ("zero diagonal", A_zero_diag)]:
    Z_rad = rng.choice([-1.0, 1.0], size=(num_samples, d))
    Z_gauss = rng.standard_normal((num_samples, d))
    tau_rad = np.einsum('si,ij,sj->s', Z_rad, M, Z_rad)
    tau_gauss = np.einsum('si,ij,sj->s', Z_gauss, M, Z_gauss)
    off = (M ** 2).sum() - (np.diag(M) ** 2).sum()
    print(name)
    print(f"  Rademacher: empirical var {tau_rad.var():9.2f}   formula {2*off:9.2f}")
    print(f"  Gaussian:   empirical var {tau_gauss.var():9.2f}   formula {2*(M**2).sum():9.2f}")
```

```
diagonal-dominant
  Rademacher: empirical var    978.64   formula    983.70
  Gaussian:   empirical var   2578.98   formula   2595.05
zero diagonal
  Rademacher: empirical var    984.84   formula    983.70
  Gaussian:   empirical var    981.17   formula    983.70
```

3 Part C: Randomized matrix multiplication

Let $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times q}$. Write a_j for the j-th **column** of A and $b_j^\top$ for the j-th **row** of B. Then

$$AB = \sum_{j=1}^{n} a_j b_j^\top,$$

a sum of n rank-one outer products.

Computing all of them is expensive, so **sample**:

- draw $J \sim p$ on $\{1, \dots, n\}$,
- form $X = \dfrac{1}{p_J}\, a_J b_J^\top$,
- average c i.i.d. copies: $\hat{C} = \dfrac{1}{c}\sum_{t=1}^{c} X^{(t)}$.

The factor $1/p_J$ is **importance weighting**: it compensates for rows that are sampled more often.

3.1 Unbiasedness of randomized matrix multiplication

X is a matrix-valued function $h(J)$ of the discrete RV J, and by LOTUS:

$$E[X] = \sum_{j=1}^{n} \frac{1}{p_j}\, a_j b_j^\top \; p_j = \sum_{j=1}^{n} a_j b_j^\top = AB.$$

By linearity, $E[\hat{C}] = \frac{1}{c}\sum_t E[X^{(t)}] = AB$ as well.

- The p_j **cancels** – the same cancellation trick as for the row-norm PMF in Kaczmarz.

- We need $p_j > 0$ wherever $a_j b_j^\top \neq 0$; otherwise those terms are never sampled and the estimator is biased.
- Unbiasedness holds for **every** valid p, so the choice of p will only affect the **variance**.

(Not on HW) Exercise 5: variance of randomized matrix multiplication

Let $X = \frac{1}{p_J} a_J b_J^\top$ as above, with $X_{ik} = \frac{a_{ij} b_{jk}}{p_j}$ conditional on $J = j$ (here a_{ij} is the i-th entry of column a_j and b_{jk} the k-th entry of row $b_j^\top$).

(a) Use LOTUS and law of total expectation to show $E[X_{ik}^2] = \sum_j \dfrac{a_{ij}^2 b_{jk}^2}{p_j}$.

(b) Deduce the variance of the (i,k) entry of the averaged estimator $\hat{C}$:

$$\mathrm{Var}(\hat{C}_{ik}) = \frac{1}{c}\left(\sum_j \frac{a_{ij}^2 b_{jk}^2}{p_j} - (AB)_{ik}^2\right).$$

3.2 The best sampling distribution

Part (b) of the exercise above can be summed to show: $E\|AB - \hat{C}\|_F^2 = \frac{1}{c}\left(\sum_j \frac{w_j^2}{p_j} - \|AB\|_F^2\right)$, where $w_j = \|a_j\|\|b_j\|$. Only the sum depends on p, so **minimize** $\sum_j w_j^2/p_j$.

Apply Cauchy-Schwarz, the same trick as in the collision fact:

$$\left(\sum_j w_j\right)^2 = \left(\sum_j \frac{w_j}{\sqrt{p_j}} \cdot \sqrt{p_j}\right)^2 \leq \left(\sum_j \frac{w_j^2}{p_j}\right)\left(\sum_j p_j\right) = \sum_j \frac{w_j^2}{p_j}.$$

Equality holds exactly when $\frac{w_j}{\sqrt{p_j}} \propto \sqrt{p_j}$, i.e. when

$$\boxed{p_j = \frac{\|a_j\|\|b_j\|}{\sum_l \|a_l\|\|b_l\|}}, \qquad \text{giving} \qquad E\|AB - \hat{C}\|_F^2 = \frac{1}{c}\left(\left(\sum_j w_j\right)^2 - \|AB\|_F^2\right).$$

Compared with uniform sampling

For $p_j = 1/n$ the sum is $n\sum_j w_j^2 \geq (\sum_j w_j)^2$ (Cauchy-Schwarz again), with equality only when **all w_j are equal**. When the w_j are very unequal, uniform sampling wastes most draws on unimportant terms, and optimal sampling can be far better.

Connection to Kaczmarz. With $B = A^\top$ we get $b_j = a_j$, so $w_j = \|a_j\|^2$ and the optimal PMF is $p_j \propto \|a_j\|^2$: the same squared-norm weighting as the row-norm PMF in RK. (Here a_j is a column of A, i.e. a row of $A^\top$.) In both algorithms, **sample in proportion to how much each piece contributes**.